



OBSERVER PATTERN

Event-Driven Order System (Order Management)

1. Mục tiêu bài học

Sau buổi học này, sinh viên có thể:

- Hiểu vấn đề coupling khi xử lý nhiều hành động sau một sự kiện
- Giải thích được Observer Pattern và bản chất Event-Driven
- Áp dụng Observer để:
 - Gửi email, Cập nhật kho
 - Ghi log, Gửi notification
 - Mà không sửa OrderService
- Nhận ra Observer là nền tảng của: Domain Event, Microservices

2. Đặt vấn đề – Before Observer

2. Đặt vấn đề – Code mùi 🐛

✗ Vấn đề thiết kế:

Vi phạm SRP & OCP

Mỗi chức năng mới → phải sửa OrderService

Tight coupling với Email, Inventory...

```
public class OrderService
{
    public void PlaceOrder(Order order)
    {
        SaveOrder(order);
        SendEmail(order);           // Coupling
        UpdateInventory(order);     // Coupling
        LogOrder(order);            // Coupling
        NotifyAdmin(order);         // Coupling
    }
}
```

3. Tư duy Event-Driven

👉 Thay vì:

“OrderService phải biết ai cần làm gì”

👉 Hãy nghĩ:

“OrderService chỉ phát sự kiện – ai quan tâm thì tự xử lý”

📌 Đây chính là Observer Pattern

4. Observer Pattern – Khái niệm

Định nghĩa (GoF):

Observer Pattern cho phép một đối tượng (Subject) thông báo tự động cho nhiều đối tượng khác (Observers) khi trạng thái của nó thay đổi.

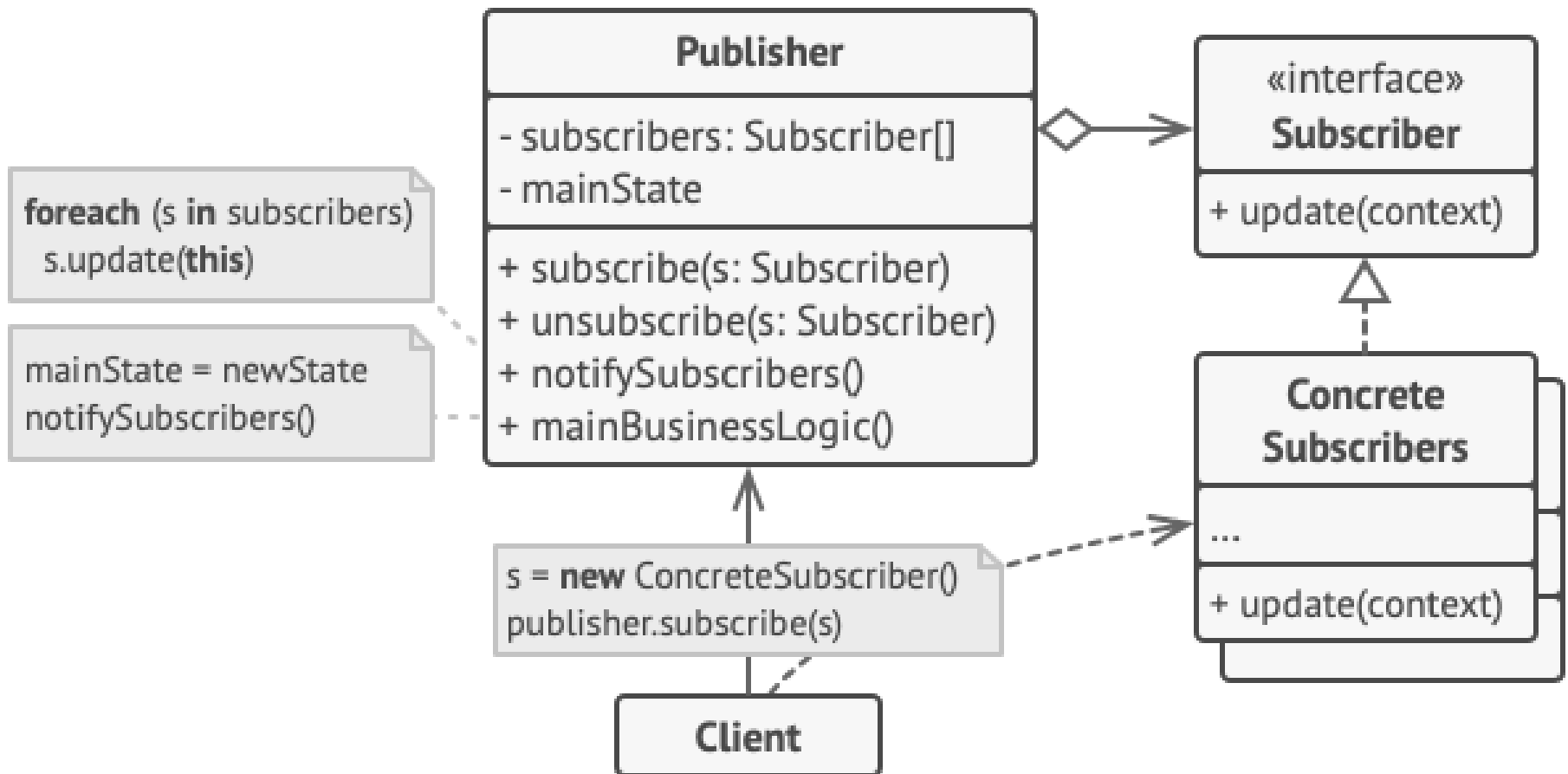
 Key words:

Subject (Publisher)

Observer (Subscriber)

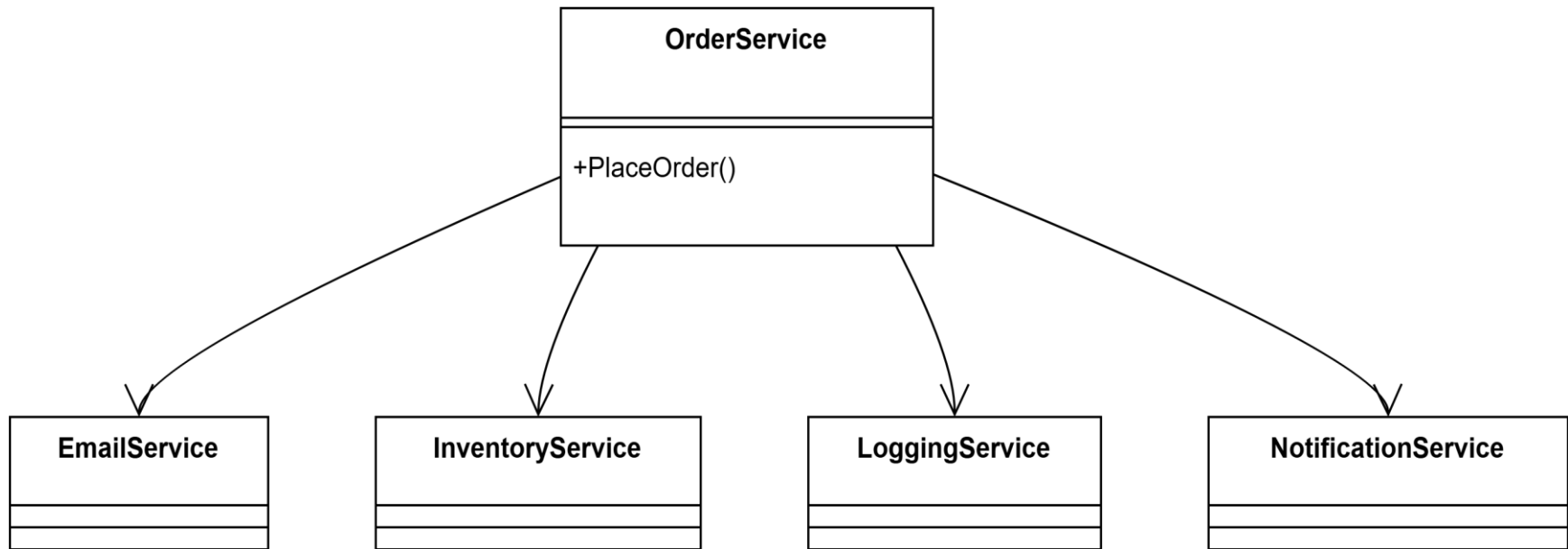
Notification / Event

5. Cấu trúc Observer Pattern



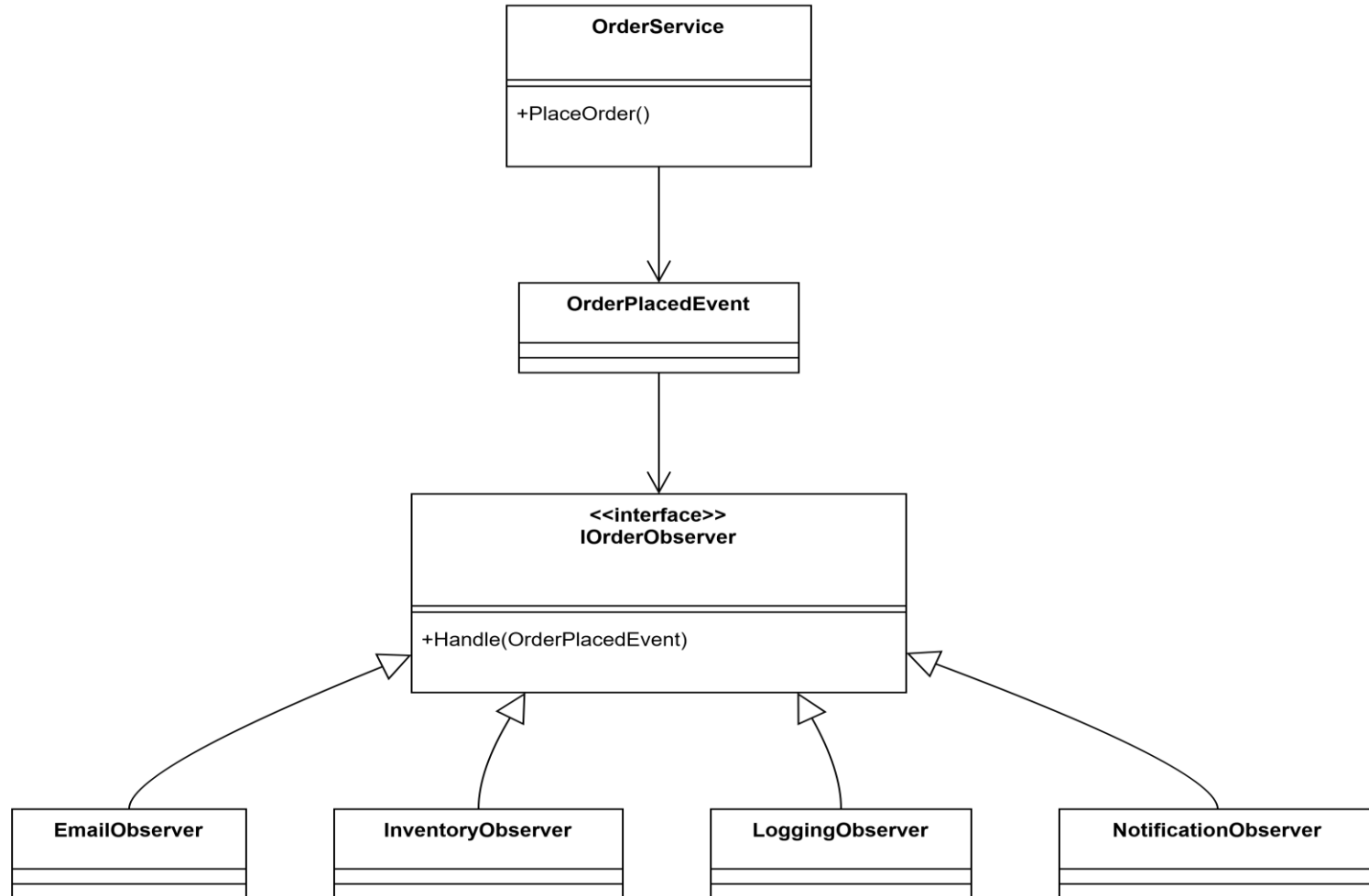
7. UML – BEFORE (Không Observer)

✗ OrderService phụ thuộc trực tiếp vào tất cả:



8. UML – AFTER (Event-Driven)

✚ OrderService không biết observer cụ thể nào:



9 & 10. Định nghĩa Event & Interface

9. Định nghĩa Event

```
public class OrderPlacedEvent {  
    public int OrderId { get; }  
    public decimal TotalAmount { get; }  
    public OrderPlacedEvent(int id, decimal total) {  
        OrderId = id; TotalAmount = total;  
    }  
}
```

10. Observer Interface

```
public interface IOrderObserver {  
    void Handle(OrderPlacedEvent orderEvent);  
}
```

11. Các Observer cụ thể

11. Các Observer cụ thể

```
public class EmailObserver : IOrderObserver {  
    public void Handle(OrderPlacedEvent e)  
        => Console.WriteLine($"Email for order {e.OrderId}");  
}  
  
public class InventoryObserver : IOrderObserver {  
    public void Handle(OrderPlacedEvent e)  
        => Console.WriteLine($"Update Inventory {e.OrderId}");  
}
```

12. Subject (Quản lý đăng ký)

12. Subject (Event Publisher)

```
public class OrderEventPublisher {  
    private readonly List<IOrderObserver> _observers = new();  
    public void Subscribe(IOrderObserver observer) {  
        _observers.Add(observer);  
    }  
    public void Publish(OrderPlacedEvent orderEvent) {  
        foreach (var observer in _observers) {  
            observer.Handle(orderEvent);  
        }  
    }  
}
```

13. OrderService (Clean Code)

✂ Không còn logic gửi mail, kho hàng ở đây nữa.

13. OrderService (Sau refactor)

```
public class OrderService {  
    private readonly OrderEventPublisher _publisher;  
  
    public OrderService(OrderEventPublisher publisher) {  
        _publisher = publisher;  
    }  
  
    public void PlaceOrder(Order order) {  
        SaveOrder(order); // Core logic  
  
        // Chỉ phát sự kiện, không quan tâm ai nhận  
        var ev = new OrderPlacedEvent(order.Id, order.Total);  
        _publisher.Publish(ev);  
    }  
}
```

14. So sánh Before / After

Tiêu chí	Before	After
Coupling	Cao (Tight)	Thấp (Loose)
SRP	Vi phạm	Tuân thủ
OCP	Sửa code khi thêm feature	Thêm class Observer mới
Test	Khó mock dependencies	Dễ unit test

15. Observer vs Strategy

Tiêu chí	Strategy	Observer
Mục đích	Chọn 1 thuật toán	Phản ứng sự kiện
Số lượng	1-1 (thường là vậy)	1-Many (1 sự kiện, nhiều xử lý)
Cơ chế	Runtime decision	Event notification

16. Liên hệ kiến trúc thực tế

Observer Pattern chính là nền tảng của:

1. Domain Events (DDD)
2. Message Queue (RabbitMQ, Kafka)
3. ASP.NET Core Events
4. Microservices Communication

17. Bài tập thực hành

Bài tập 1 (Cơ bản):

Thêm SmsObserver và đăng ký vào Publisher.

Đảm bảo không sửa OrderService.

Bài tập 2 (Nâng cao):

Chuyển cơ chế Publish sang Async (Task).

Đảm bảo nếu 1 Observer lỗi, các Observer khác vẫn chạy.

Bài tập 3 (Tư duy kiến trúc):

Vẽ sơ đồ nếu tách EmailService ra thành một Microservice riêng, Observer sẽ thay đổi thế nào? (Gợi ý: Message Queue)

18. Kết luận

“Đừng để một class làm mọi thứ. Hãy để nó phát sự kiện, và để thế giới xung quanh tự phản ứng.”

 Next Step: Clean Architecture + Domain Events